

CMP-4501 新增訂單時，上傳附件超過限制需報錯（前端） — 測試結果報告

版本紀錄

版本	日期	修訂內容	修訂者
1.0	2026-06-10	初版測試設計（建立測項清單，待 UAT 實測）	Raelynn
1.1	2026-06-10	UAT 實測 TC-01 / 02 / 05 通過，回填實際結果與截圖	Raelynn
1.2	2026-06-18	補上「二、根因分析與解決方法」章節，後續章節順移編號	Raelynn

一、測試資訊

項目	內容
需求描述	新增訂單時上傳附件超過數量限制 (>10)，後端 API 報錯，但前台顯示「add-order [object Object]」。需改為顯示後端回傳的錯誤訊息（如「上傳的檔案數量不可超過 XX 個」）。
測試環境	CMP UAT：https://cmp-uat-100.metaage.com.tw
測試帳號	raelynnlin@metaage.com.tw（統編 16428796）
測試身份	業務（新增訂單）
測試工具	agent-browser (Chrome) + XHR override（偽造 POST /order 錯誤回應）
驗證方式	觀察 notify 通知內容是否為後端錯誤訊息（非 [object Object]）；TC-03 / 04 以 XHR override 偽造回應模擬各類錯誤型別
受測檔案	orders/detail/detail.component.ts、share/services/order.service.ts
測試者	Raelynn
測試日期	2026-06-10

二、根因分析與解決方法

現象

新增訂單時，子單「附加檔案列表」上傳附件數量超過限制 (>10 個)，後端 API 回傳錯誤，但前台通知卻顯示 `add-order [object Object]`，使用者看不到真正的失敗原因（例如「上傳的檔案數量不可超過 XX 個」）。

根因

問題出在 `detail.component.ts` 的 `addOrder()` error handler，搭配 `order.service.ts` `postOrderWithFullError()` 的丟錯型別，共有兩個成因：

根因 1 — 錯誤物件被當作通知內容字串化

原 handler 以 `this.notify.error('add-order', err)` 將整個 `err` 物件當作通知「內容」傳入；NG-ZORRO 通知元件會對該參數做字串化，物件即被轉成 `[object Object]`，後端真正回傳的訊息因此被吞掉。

根因 2 — handler 自身在無 `code` 的錯誤上拋 `TypeError`

handler 第一行 `if (err.code.indexOf('bpm_fail') !== -1)` 假設 `err.code` 必為字串。但 `postOrderWithFullError()` 的 `catchError` 有兩條丟出路徑 ([order.service.ts:70-79](#))：

後端回應	丟出內容	err.code
<code>error.error?.info.code</code> 存在 (如 bpm 流程錯誤)	<code>throwError(() => error.error.info)</code> (物件，含 <code>code / message</code>)	有值 (字串)
無 <code>info.code</code> (附件數量超限即屬此類)	<code>throwError(() => new Error(errorMessage))</code> (標準 <code>Error</code> ，無 <code>code</code>)	<code>undefined</code>

附件數量超限走的是第二條路徑，`err` 是無 `code` 的 `Error`；此時 `err.code.indexOf(...)` 等於 `undefined.indexOf(...)`，直接拋 `TypeError`，handler 中斷，連錯誤通知都顯示不出來。

解決方法 (修正提交 11bca68b)

於 `detail.component.ts` `addOrder()` error handler：

- 1. 型別防護：bpm_fail 判斷前加上 `typeof err?.code === 'string'`，避免在無 `code` 的 `Error` 上呼叫 `indexOf` 而拋 `TypeError`。
- 2. 顯示後端訊息：將 `notify.error('add-order', err)` 改為取 `err?.message || err?.error?.message || this.translate.instant('error message')` 作為通知內容文字，避免物件被字串化成 `[object Object]`，使後端的數量超限訊息能正確呈現。

```
// 修正後
error: (err) => {
  console.error('add-order', err);
  // bpm 流程錯誤：err.code 可能不存在，需先檢查型別
  if (typeof err?.code === 'string' && err.code.indexOf('bpm_fail') !== -1) {
    this.notify.error(this.translate.instant('error message'), err.message, { nzDuration: 8000 });
    // ... (1 秒後 finalizeOrder 導頁)
    return;
  }
  this.ui.isLoading = false;
  // 顯示後端回傳訊息，避免帶入錯誤物件顯示成 [object Object]
  const errorMessage = err?.message || err?.error?.message || this.translate.instant('error message');
  this.notify.error(this.translate.instant('error message'), errorMessage, { nzDuration: 8000 });
},
```

對應驗證：根因 1 由 TC-01 / 02 驗證 (顯示後端訊息、非 `[object Object]`)；根因 2 由 TC-04 驗證 (無 `code` 的 `Error` 不再拋 `TypeError`)；TC-03 則確保 bpm_fail 既有分支不受防護影響。

三、測試案例總覽

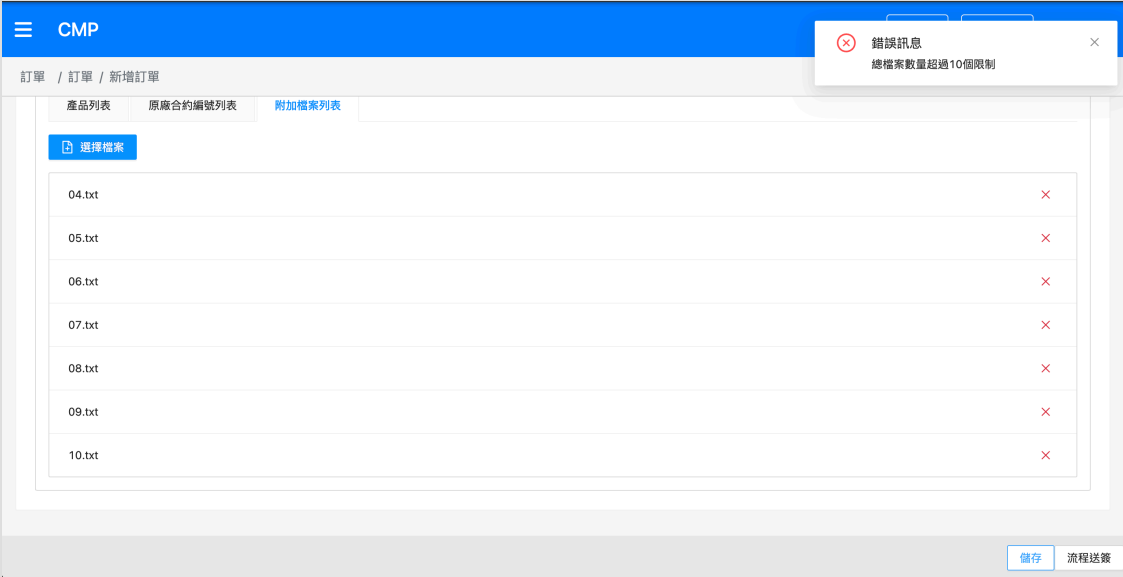
編號	群組	測項	結果
TC-01	A 新增訂單	附件超過限制 + 儲存草稿 → 顯示後端訊息（非 [object Object]）	✔ Pass
TC-02	A 新增訂單	附件超過限制 + 流程送簽(skipDraft) → 顯示後端訊息	✔ Pass
TC-03	A 新增訂單	bpm_fail 類錯誤 → 顯示 err.message + 1 秒後導頁	✔ Pass
TC-04	A 新增訂單	一般 Error（無 code）→ 不拋 TypeError、正常顯示訊息	✔ Pass
TC-05	A 新增訂單	正常新增訂單成功（回歸）	✔ Pass

共通檢查：所有錯誤通知內容皆不得出現 [object Object]；error handler 本身不得因存取 err.code 等屬性而拋出例外（導致連通知都不顯示）。**重現方式註記：**TC-01 / 02 / 05 由 UI 自然操作觸發；TC-03 / 04 需以 XHR override 偽造 POST /order 回應，模擬特定錯誤型別（bpm_fail code、無 code 的錯誤），詳見附錄 B。

四、測試案例

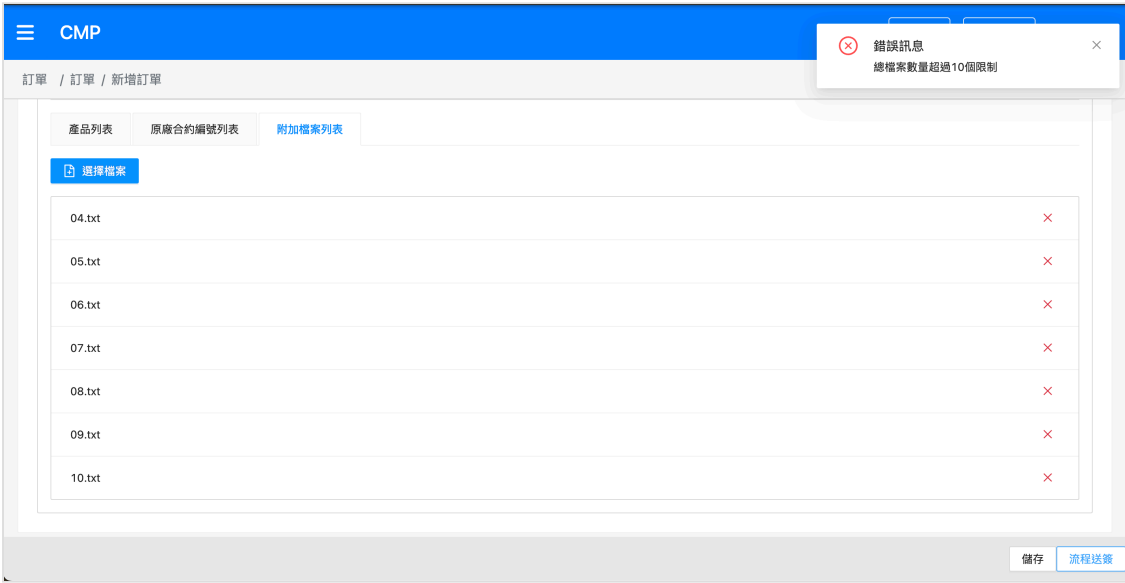
A 群組、新增訂單錯誤訊息（detail.component.ts addOrder）

TC-01 — 附件超過限制 + 儲存草稿

項目	內容
前置	已登入、業務身份、AWS 新增訂單頁、必填欄位已齊（見測試準備 4）、已備妥 12 個小檔
步驟	① 切到子單「附加檔案列表」分頁 ② 第 1 批 選 6 個檔上傳（應成功，前端不擋）③ 第 2 批 再選 6 個檔上傳（應成功，本機累計 12）④ 點頁面右下「儲存」
預期	訂單 POST 後，跳出錯誤通知，內容為 後端回傳訊息 （如「上傳的檔案數量不可超過 10 個」）， 非 add-order [object Object]；ui.isLoading 解除、停留於新增頁可續編輯
實際	分 2 批各 6 個（合計 12）上傳成功；點「儲存」POST 後，前端正確顯示後端回傳之附件數量超限錯誤訊息， 未出現 [object Object]；isLoading 解除、停留於新增頁可續編輯。（見截圖）
截圖	

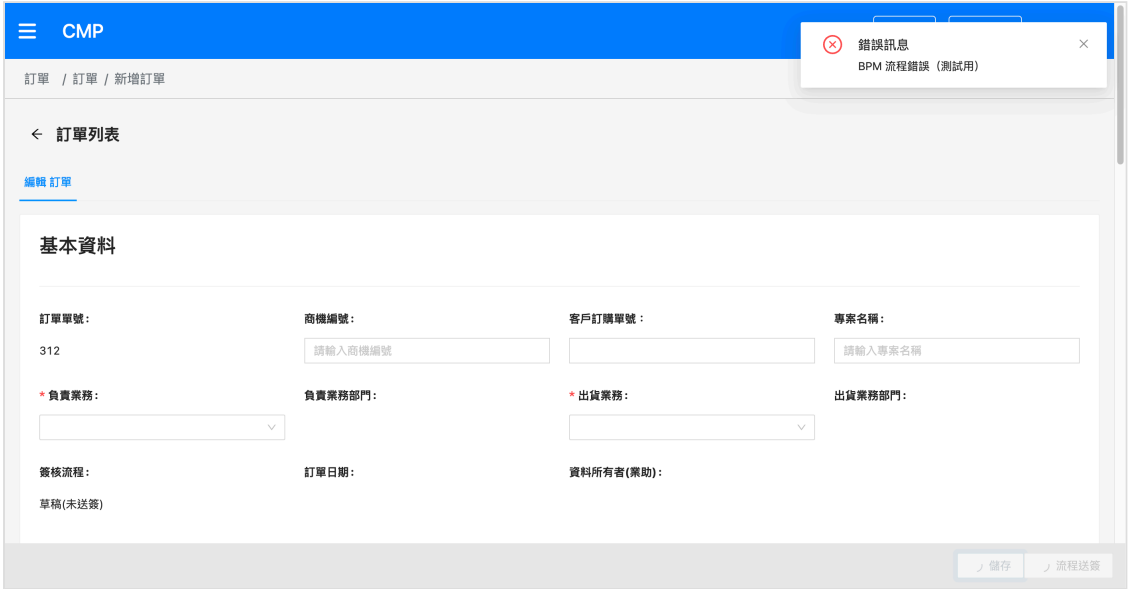
結果	✔ Pass
----	--------

TC-02 — 附件超過限制 + 流程送籤（skipDraft）

項目	內容
前置	同 TC-01（同一張單或重建一張，附件分批上傳達 12 個）
步驟	①～③ 同 TC-01 分批上傳達 12 個 ④ 改點頁面右下「流程送籤」
預期	同 TC-01：顯示後端錯誤訊息且非 <code>[object Object]</code> ； <code>ui.isLoading</code> 解除（不會推進到 PM 審核）
實際	分批上傳達 12 個後點「流程送籤」，POST 後正確顯示後端附件數量超限錯誤訊息，未出現 <code>[object Object]</code> ； <code>isLoading</code> 解除、未推進到 PM 審核。（見截圖）
截圖	
結果	✔ Pass

TC-03 — bpm_fail 類錯誤

項目	內容
前置	新增 AWS 訂單頁，已注入 XHR override（附錄 B），令 <code>POST .../order</code> 回傳 <code>{ info:{ code:'bpm_fail_test', message:'BPM 流程錯誤（測試用）' } }</code> 、status 400
步驟	① 點「儲存」送出

預期	以標題「錯誤訊息」顯示 <code>err.message</code> (8 秒)，1 秒後執行 <code>finalizeOrder</code> 並導頁； <code>isLoading</code> 解除
實際	XHR override 攔截 POST (<code>__fakeFired=1</code>)；通知顯示「錯誤訊息 / BPM 流程錯誤 (測試用)」(即 <code>err.message</code>)，非 <code>[object Object]</code> ；約 1 秒後自動導頁回 <code>/main/orders</code> (<code>finalizeOrder</code>)。走 <code>bpm_fail</code> 分支正確。(見截圖)
截圖	
結果	✅ Pass

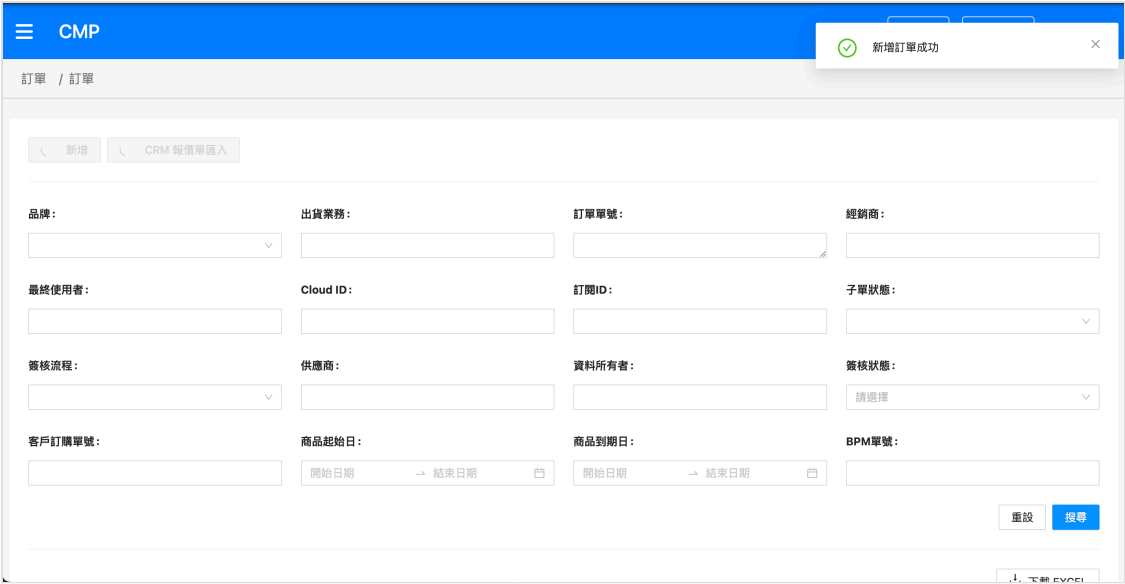
TC-04 — 一般 Error (無 code)

項目	內容
前置	新增 AWS 訂單頁，已注入 XHR override，令 <code>POST .../order</code> 回傳 <code>{ info:{}, message:'伺服器發生錯誤 (測試用)' }</code> 、status 500 (無 <code>info.code</code> ，走 <code>new Error(errorMessage)</code> 路徑)
步驟	① 點「儲存」送出
預期	不因 <code>err.code.index0f</code> 拋 <code>TypeError</code> ；以「錯誤訊息」標題顯示 <code>err.message</code> / fallback，畫面不出現 <code>[object Object]</code> ； <code>isLoading</code> 解除
實際	XHR override 攔截 POST (<code>__fakeFired=1</code>)；通知顯示「錯誤訊息 / 伺服器發生錯誤 (測試用)」(即 <code>err.message</code>)，非 <code>[object Object]</code> ；未拋 <code>TypeError</code> (handler 正常完成)、未導頁 (停留 <code>/main/orders/add</code> ，非 <code>bpm_fail</code> 分支)。(見截圖)

截圖	
結果	✅ Pass
備註	本案為本次新增 <code>typeof err?.code === 'string'</code> 防護的重點驗證項——舊版 <code>err.code.indexOf(...)</code> 會在此拋 <code>TypeError</code> ，修正後正常顯示訊息

TC-05 — 正常新增訂單成功（回歸）

項目	內容
前置	AWS 新增訂單頁、必填齊全、附件數量在限制內（≤10 個，或不傳附件）
步驟	① 建立合法 AWS 訂單（同準備 4）② 附件 ≤10 個 ③ 點「儲存」（或「流程送簽」）
預期	顯示「新增訂單成功」，正常進入後續流程（ <code>finalizeOrder</code> ），無錯誤通知、無 <code>[object Object]</code>
實際	合法 AWS 訂單、附件在限制內，點「儲存」後顯示「新增訂單成功」，正常進入後續流程，無錯誤通知、無 <code>[object Object]</code> 。（見截圖）

截圖	
結果	✅ Pass

五、測試結果總覽

群組	TC 數	Pass	Fail	Blocked	待測	備註
A 新增訂單	5	5	0	0	0	TC-01~05 全數 Pass
總計	5	5	0	0	0	全數通過

六、缺陷紀錄

無。TC-01~05 全數通過，未發現缺陷。

七、附錄

附錄 A — XHR override (偽造 POST /order 回應，用於 TC-03 / 04)

於新增訂單頁注入一次 (SPA 導頁不會清除，整頁重載才需重裝)。攔截 `POST .../order`，當 `window.__fakeOrderError` 有值時不打後端、改回傳偽造的錯誤回應，以觸發 `addOrder` 的 error handler。

```
agent-browser eval "  
(function(){  
  if (window.__xhrPatched) return 'already';  
  window.__xhrPatched = true;  
  const RS = XMLHttpRequest.prototype.send, RO = XMLHttpRequest.prototype.open;  
  XMLHttpRequest.prototype.open = function(m,u){ this.__m=(m||'').toUpperCase(); this.__u=u||''; retur  
  XMLHttpRequest.prototype.send = function(b){  
    if (this.__m==='POST' && /\order(?:|$)/.test(this.__u) && window.__fakeOrderError){
```

```

const s=this, f=window.__fakeOrderError;
Object.defineProperty(s, 'readyState', {get:()=>4, configurable:true});
Object.defineProperty(s, 'status', {get:()=>f.status, configurable:true});
Object.defineProperty(s, 'response', {get:()=>f.body, configurable:true});
Object.defineProperty(s, 'responseText', {get:()=>JSON.stringify(f.body), configurable:true});
s.getAllResponseHeaders=()=>'content-type: application/json\r\n';
s.getResponseHeader=h=>/content-type/i.test(h)?'application/json':null;
window.__fakeFired=(window.__fakeFired||0)+1;
setTimeout(()=>{ s.dispatchEvent(new Event('load')); s.dispatchEvent(new Event('loadend')); },10
return;
}
return RS.apply(this,arguments);
};
return 'patched';
})();
"

```

注意：postOrderWithFullError 走 Angular HttpClient (responseType=json，內部使用 XHR)，故偽造的 response 直接回物件即可被 HttpClient 當成已解析 body。

附錄 B — 各測項偽造回應設定

設定 window.__fakeOrderError 後點「儲存」觸發：

測項	window.__fakeOrderError	走的分支 / 目的
TC-03	{ status:400, body:{ info:{ code:'bpm_fail_test', message:'BPM 流程錯誤 (測試用)' } } }	bpm_fail 分支：顯示 err.message + 1 秒後導頁
TC-04	{ status:500, body:{ info:{}, message:'伺服器發生錯誤 (測試用)' } }	new Error() 分支：驗證 typeof err?.code 防護，顯示 err.message 不拋例外

TC-04 的 body 需含 info:{} (空物件)，以避開 order.service.ts 中 error.error?.info.code 在 info 不存在時的存取例外 (屬另一潛在問題，非本次受測範圍)。

附錄 C — 受測程式修正對照

檔案	修正點	對應 TC
detail.component.ts addOrder error handler	顯示 err.message (取代原 notify.error('[add-order]', err)); err.code 加 typeof err?.code === 'string' 防護	TC-01 ~04
order.service.ts postOrderWithFullError	直接走 HttpClient 並丟出原始物件 (error.error.info) 之來源——即 [object Object] 的成因	全部